**Program:** BS Computer Science
**Semester:** Fall 2025
**Credit Hour(s):** 1
**Pre-requisite(s):** EE1005 Digital Logic Design
**Instructor:** Engr. Muhammad Hassaan Bin Shoukat – hassaan.shoukat@nu.edu.pk

# Lab # 13: String Instructions & Interrupts

## 1. Objectives

- Understand and utilize the String Instructions: STOS, LODS, MOVS, SCAS.
- Manipulate the Direction Flag (DF) using CLD and STD to control processing direction.
- Use REP prefixes (REP, REPE, REPNE) to process data blocks without explicit loop instructions.
- Understand the roles of DS:SI (Source Index) and ES:DI (Destination Index).

## 2. String Instructions

String instructions allow processing blocks of memory efficiently without writing explicit loops for fetching and storing.

- **REP:** Repeats the instruction CX times
- **CLD (Clear Direction Flag):** (Clear) processes forward (left-to-right)
- **STD (Set Direction Flag):** (Set) processes backward
- **STOSB/STOSW (Store String):** Store AL/AX into [ES:DI], increment DI by 2
- **LODSB/LODSW (Load String):** Load string from [DS:SI] into AL/AX
- **MOVSB/MOVSW (Move String):** Move block from [DS:SI] to [ES:DI]
- **SCASB/SCASW (Scan String):** Compares AL/AX with byte at [ES:DI] and updates flags
- **REPE, REPNE:** Repeat prefixes

### 2.1. STOS (Store)

**Example 1:** Fill 10 Screen Cells with '*'

```
[org 0x0100]

mov ax, 0B800h          ; video memory segment
mov es, ax

mov di, 0               ; start at top-left of screen
mov ax, '*' + (07h << 8)    ; '*' with light-grey attribute
mov cx, 10              ; fill 10 screen cells (10 characters)

cld                     ; ensure forward direction
rep stosw               ; fill CX times with AX
```

```
mov ax, 4C00h
int 21h
```

**Example 2:** The fastest way to clear the screen using REP STOSW.

```
[org 0x0100]
    mov ax, 0xb800
    mov es, ax
    xor di, di          ; DI = 0 (Top Left)

    mov ax, 0x0720      ; 0x20 = Space, 0x07 = Normal Attribute
    mov cx, 2000    ; Whole screen (80 cols*25 rows = 2000 chars)

    cld                 ; Clear Direction Flag (Forward)
    rep stosw           ; Repeat 'Store Word' 2000 times
                       ; effectively filling screen with spaces
mov ax, 0x4c00
int 0x21
```

## 2.2. LODS (Load and Print Each Character)

**Example 3:** Reads characters from a string using *LODSB*.

```
[org 0x0100]

mov ax, 0B800h
mov es, ax

mov si, msg         ; lodsb will read characters from [DS:SI]
mov cx, 11
mov di, 0

cld             ; auto-increment
next_char:
    lodsb       ; AL = [SI], SI++
    mov ah, 07h
    mov [es:di], ax
    add di, 2
    loop next_char
mov ax, 4C00h
int 21h
msg: db "Hello World",0
```

**Example 4:** Loads bytes from an array and computes the sum.

```
[org 0x0100]
mov si, nums
mov cx, 5
```

```
xor ax, ax

cld
read_loop:
    lodsb       ; load next byte into AL
    add bl, al
    loop read_loop

mov ax, 4C00h
int 21h

nums: db 4, 8, 12, 16, 20
```

## 2.3. MOVS (Move String Instruction)

**Example 5:** Copy video row

```
[org 0x0100]

mov ax, 0B800h
mov es, ax
mov ds, ax

mov si, 0         ; row 0
mov di, 160       ; row 1
mov cx, 80        ; 80 words

cld
rep movsw            ; Copy word from row 0 to row 1

mov ax, 4C00h
int 21h
```

**Example 6:** String Copy.

```
[org 0x0100]
    jmp start

source: db 'Hello'
dest:   db '     '       ; Empty buffer
start:
    mov ax, ds
    mov es, ax   ; For MOVS, ES must point to destination segment

    mov si, source      ; Source Index
    mov di, dest        ; Destination Index
    mov cx, 5           ; Length to copy
```

```
    cld                 ; Process forward
    rep movsb           ; Move Byte string.
    ; Copies [SI] to [DI], inc SI, inc DI, dec CX
    ; 'dest' now contains "Hello"
mov ax, 0x4c00
int 0x21
```

## 2.4. SCAS (Scan String Instruction)

**Example 7:** Find character 'A'

```
[org 0x0100]

mov di, str1
mov cx, 12
mov al, 'A'

cld
repne scasb      ; search until AL == [DI]
; ZF = 1 if found
mov ax, 4C00h
int 21h

str1: db "BBAACCDDEEFF",0
```

**Example 8:** Find first non-zero byte

```
[org 0x0100]
    mov ax, ds
    mov es, ax          ; SCASB always uses ES:DI

    mov di, buffer      ; start of buffer
    mov cx, 20          ; scan 20 bytes
    mov al, 0     ; looking for NON-zero(repe=repeat while zero)

    cld                 ; forward direction
    repe scasb          ; stop when byte != 0

    dec di              ; point DI back to actual byte found
    ; (SCASB increments DI before checking result)
    mov al, [di]        ; load found byte
    add al, 30h         ; convert to ASCII
    mov ah, 0Eh
    int 10h             ; print the digit
mov ax, 4C00h
int 21h
buffer: db 0,0,0,0,0,  0,0,0,0,0,   0,0,0,0,5,  7,0,0,0,0
```

## 3. Interrupts

Interrupts are asynchronous events generated outside the processor that require immediate attention. The 8088 divides them into two classes:

1. **Hardware Interrupts:** Generated by external hardware (e.g., keyboard, timer).
2. **Software Interrupts:** Generated by the ***INT*** instruction within the program. They act as an "*extended far call*" mechanism.

### 3.1. The Interrupt Mechanism (Microcode)

When an interrupt occurs (e.g., INT N):

1. The processor completes the current instruction.
2. It pushes the **FLAGS** register onto the stack.
3. It pushes the current **CS** and **IP**.
4. It clears the **Trap Flag (TF)** and **Interrupt Flag (IF)** to disable further interrupts.
5. It fetches the new CS and IP from the **Interrupt Vector Table (IVT)**.

### 3.2. The Interrupt Vector Table (IVT)

- **Location:** Fixed at physical address 0x00000 (0000:0000)
- **Size:** 1024 bytes (*256 interrupts x 4 bytes per entry*)
- **Structure:** Each entry is 4 bytes. The first two bytes are the **Offset (IP)**, and the next two are the **Segment (CS)**
- Changing a vector = *hooking an interrupt*

"Hooking" means changing the address in the IVT to point to our own function (Interrupt Service Routine - ISR) instead of the system default. We will modify the vector for INT 0 so that if a division by zero occurs, *our* code executes.

***Note:*** *The DIV instruction automatically generates INT 0 if the quotient is too large. Unlike normal interrupts, INT 0 returns to the* ***same*** *instruction that caused it, not the next one. This often causes an infinite loop if not handled correctly.*

### 3.3. Reserved Interrupts

Intel reserves the first 5 interrupts for dedicated functions:

- **INT 0:** Divide by Zero
- **INT 1:** Single Step (Trap)
- **INT 2:** Non-Maskable Interrupt (NMI)
- **INT 3:** Debug Breakpoint
- **INT 4:** Arithmetic Overflow

**Example 13:** Software Interrupt – INT 21h (DOS). Print a string using DOS interrupt

```
[org 0x0100]
     mov dx, msg
```

```
     mov ah, 09h       ; print string
     int 21h           ; software interrupt
mov ax, 4C00h
int 21h
msg: db 'Hello from INT 21h!$', 0
```

**Example 14:** BIOS Interrupt – INT 10h (Video Services). Print characters using BIOS teletype

```
[org 0x0100]
     mov ah, 0Eh       ; BIOS teletype function
     mov al, 'A'
     int 10h

     mov al, 'B'
     int 10h
mov ax, 4C00h
int 21h
```

**Example 15:** Keyboard Input – INT 16h. Wait for user to press a key and display it

```
[org 0x0100]
     mov ah, 00h       ; wait for key
     int 16h           ; result in AL

     mov ah, 0Eh       ; print AL through BIOS
     int 10h
mov ax, 4C00h
int 21h
```

**Example 16:** System Time – INT 1Ah. Read system clock ticks

```
[org 0x0100]
     mov ah, 00h       ; read system time
     int 1Ah           ; CX:DX = # of ticks since midnight

     ; print high byte of DX (just for demonstration)
     mov al, dh
     add al, 30h

     mov ah, 0Eh
     int 10h
mov ax, 4C00h
int 21h
```

**Example 17:** Software Interrupt Triggering Error (INT 0 – Division by Zero). Division error interrupt example.

```
[org 0x0100]
```

```
    mov ax, 100
    mov bl, 0
    div bl      ; generates INT 0 (division error)
mov ax, 4C00h
int 21h
```

## 4. Lab Tasks

**Task 1:** Using **STOSW** with **REP**, write a program that:

- Fills the **first full row** of the screen with the character 'X' in attribute **1Eh** (yellow on blue).
- You must use:
  a) mov ax, 0B800h → ES
  b) cld
  c) rep stosw

**Task 2:** Write a program that:

- Copies **row 5** to **row 10** on the screen.
- The program must use:
  a) *rep movsw*
  b) DS:SI as source
  c) ES:DI as destination

**Task 3:** Write a program that:

- Copies **row 0** to **row 1**, but using **backward direction**.
- You must use *std* & *rep movsw*
- Start from the *end of the row* and move backward

**Task 4:** Write a program that:

- Reads characters from the string, `msg: db "COAL LAB 13",0`
- Uses *lodsb*, *cld* & a loop to write each character to video memory
- Print the string starting at row 3, column 20

**Task 6:** Write a program that:

- Defines a pointer: `vptr: dw 0, 0B800h`
- Uses les di, [vptr]
- Writes character 'H' with attribute 0Fh at the video memory location ES:DI using *stosw*.

**Task 7:** Write a program that:

- Saves the old vector of INT 0
- Installs a new interrupt handler at INT 0 that:
  a) Prints "Division Error!" using BIOS int 10h

b) Returns with IRET

- Perform div bl where bl=0 to trigger INT 0
- Restore original INT 0 vector before exiting

**Task 8:** Write a program that searches for the character 'a' in the string "Assembly Language".

- Use SCASB with the REPNE prefix.
- If found, save the position (offset) of the character in the DX register.

**Task 9:** Write a program that hooks **INT 60h**.

- Your ISR for INT 60h should print "Custom Interrupt Called!"
- In your main program, manually trigger this interrupt using the instruction INT 0x60.